



北京航空航天大学
BEIHANG UNIVERSITY



操作系统

王良

lwang20@buaa.edu.cn



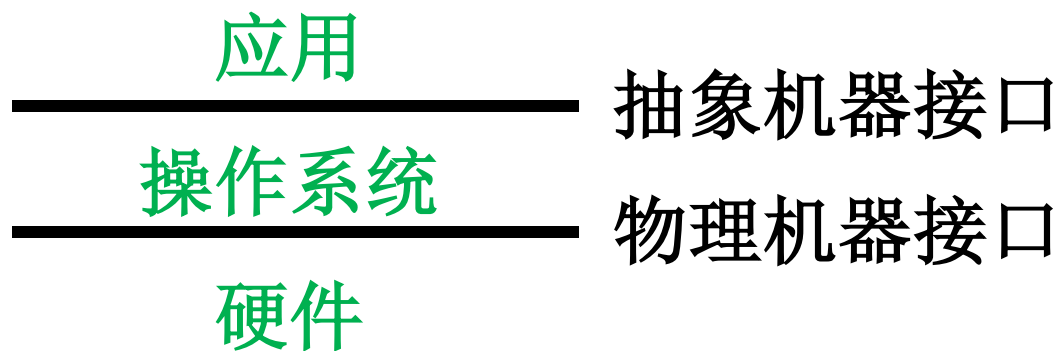
操作系统引论

- 什么是操作系统
- 为什么学习操作系统
- 怎么学习操作系统
- 操作系统简史
- **操作系统功能与特征**
- 操作系统的结构
- 计算机硬件与操作系统机制



操作系统对硬件进行抽象

- 处理器 → 进程
- 存储器 → 地址空间
- 磁盘, SSDs, ... → 文件





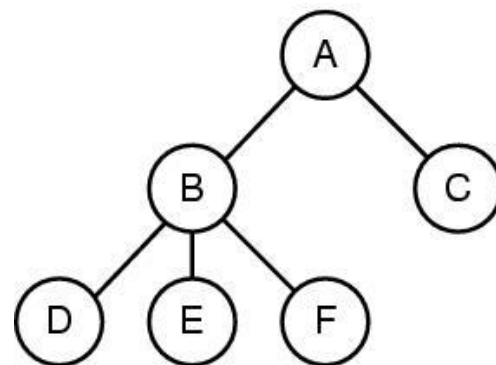
操作系统的功能

- 处理机管理
- 存储器管理（内存）
- 设备管理
- 文件管理
- 接口管理



处理机管理

- 处理机分配和运行都以进程为最基本单位，主要有以下功能
- **进程控制**：创建和撤销进程
 - 创建、撤销、状态转换
- **进程同步**：对各进程运行进行协调
 - 互斥、同步
- **进程通信**：实现进程间的信息交换
- **进程调度**：按照一定算法把处理机分配给进程

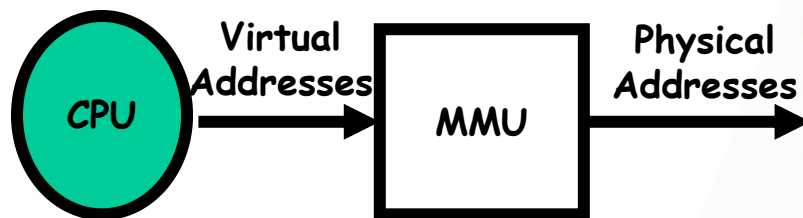


■ 任务:

- 为多道程序的并发提供良好的环境
- 便于用户使用存储器
- 提高存储器利用率
- 为尽量多的用户提供足够大的存储空间

■ 功能:

- 内存分配和回收: 静态和动态分配
- 内存保护
- 地址影射
- 内存扩充





■ 任务:

- 为用户程序分配I/O设备
- 完成用户程序请求的I/O操作
- 提高CPU和I/O设备的利用率：中断；通道
- 改善人机界面

■ 功能:

- 缓冲管理
- 设备分配
- 设备处理
- 虚拟设备功能



文件系统

- 文件存储空间的管理
- 目录管理
- 文件读、写管理
- 文件保护
- 向用户提供接口



■ 程序接口

- 系统调用

■ 用户接口

- 联机用户接口
- 脱机用户接口
- 图形用户接口



系统调用 (System call)

■ 为什么要有系统调用?

- 运行在用户空间的程序向操作系统内核请求需要更高
- 权限运行的服务, 例如I/O、申请内存等

■ 系统调用提供用户程序与操作系统之间的接口



Linux的常用系统调用

Number	Name	Description	Number	Name	Description
1	exit	Terminate process	27	alarm	Set signal delivery alarm clock
2	fork	Create new process	29	pause	Suspend process until signal arrives
3	read	Read file	37	kill	Send signal to another process
4	write	Write file	48	signal	Install signal handler
5	open	Open file	63	dup2	Copy file descriptor
6	close	Close file	64	getppid	Get parent's process ID
7	waitpid	Wait for child to terminate	65	getpgrp	Get process group
11	execve	Load and run program	67	sigaction	Install portable signal handler
19	lseek	Go to file offset	90	mmap	Map memory page to file
20	getpid	Get process ID	106	stat	Get information about file



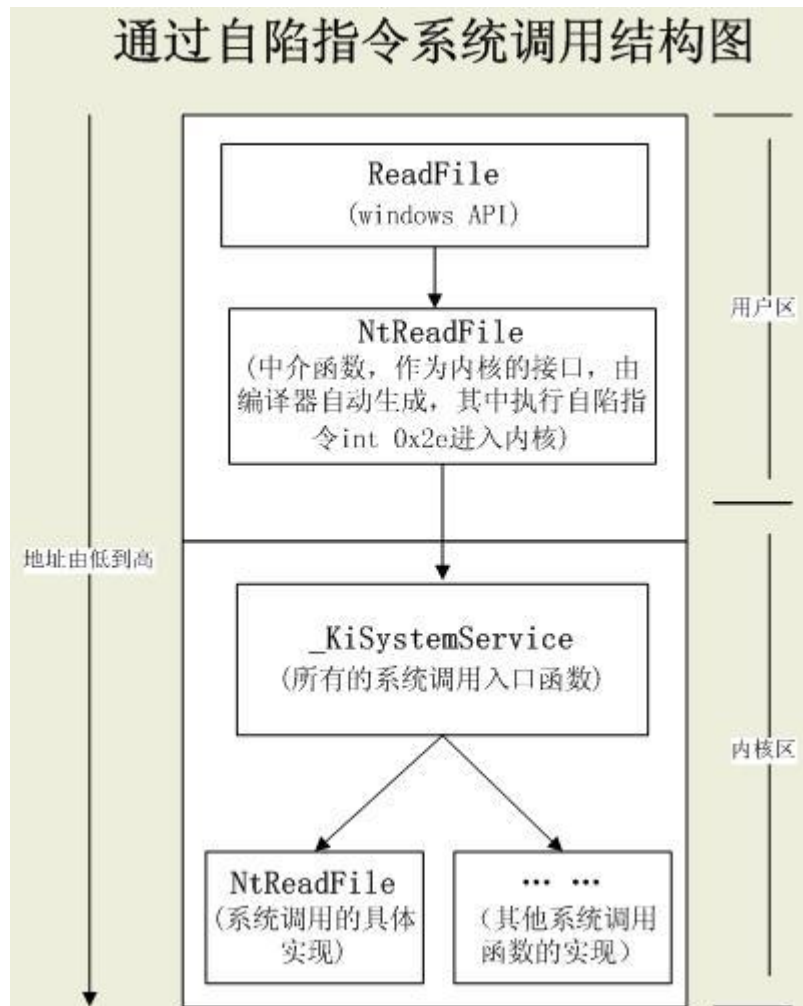
函数库调用 vs. 系统调用

函数库调用	系统调用
在所有的 ANSI C 编译器版本中，C 库函数是相同的	各个操作系统的系统调用是不同的
它调用函数库中的一段程序（或函数）	它调用系统内核的服务
与用户程序相联系	是操作系统的一个入口点
在用户地址空间执行	在内核地址空间执行
它的运行时间属于“用户时间”	它的运行时间属于“系统”时间
属于过程调用，调用开销较小	需要在用户空间和内核上下文环境间切换，开销较大
在 C 函数库 libc 中有大约 300 个函数	在 UNIX 中大约有 90 个系统调用
典型的 C 函数库调用：system fprintf malloc	典型的系统调用：chdir fork write brk;

<https://www.cnblogs.com/muahao/p/7225236.html>

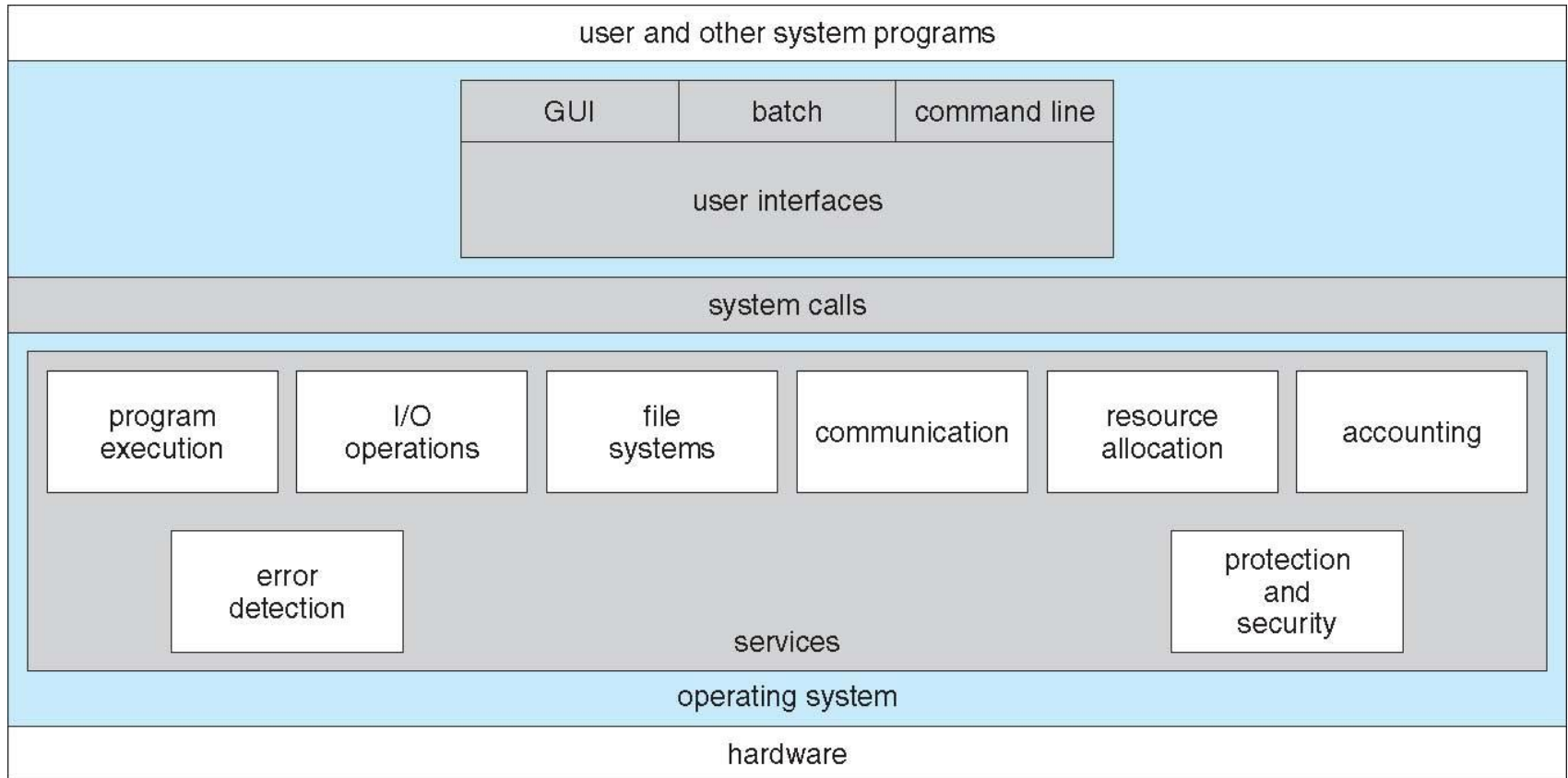
System Call执行过程

1. CPU执行int 0x2e，CPU运行状态切换为系统态；
2. 当前进程使用的堆栈，从用户态切换到系统态。堆栈原先的内容（用户态），以及用户空间的堆栈指针，被压入系统空间堆栈。
3. 从中断向量表中(Interrupt Descriptor Table)以0x2e为中断向量，开始执行系统空间中的程序。
4. 程序执行后，通过iret(中断返回)指令实现上述过程的逆过程





Operating System Services





用户操作系统接口 —— CLI

CLI or command interpreter allows direct command entry

- Sometimes implemented in kernel, sometimes by systems program
- Sometimes multiple flavors implemented – shells
- Primarily fetches a command from user and executes it
- Sometimes commands built-in, sometimes just names of programs
 - If the latter, adding new features doesn't require shell modification



用户操作系统接口 —— GUI

■ User-friendly desktop metaphor interface

- Usually mouse, keyboard, and monitor
- **Icons** represent files, programs, actions, etc
- Various mouse buttons over objects in the interface cause various actions (provide information, options, execute function, open directory (known as a **folder**))
- Invented at Xerox PARC

■ Many systems now include both CLI and GUI interfaces

- Microsoft Windows is GUI with CLI “command” shell
- Apple Mac OS X is “Aqua” GUI interface with UNIX kernel underneath and shells available
- Unix and Linux have CLI with optional GUI interfaces (CDE, KDE, GNOME)



操作系统应解决的基本问题

■ 提供解决各种冲突（资源竞争引起）的策略

- 处理机调度、进程调度、内存分配、设备分配等

■ 协调并发活动的关系（提供流程控制和避免运行结果不确定性）

- 进程之间的通信，同步与互斥

■ 保证数据的一致性

- 读写数据时，数据结构中的内容是否真实地记录了数据的实际情况
- 在分布式处理时的共享数据的不同副本是否一致

■ 实现数据的存取控制

- 共享程度、隐私程度、安全程度的控制



操作系统的特征

■ 并发

■ 共享

- 互斥共享（打印机、变量）
- 同时访问（宏观）

■ 虚拟

- 多道程序中的CPU
- SPOOLING（Simultaneous Peripheral Operations On-Line，外围设备同时联机操作）
- 虚拟存储

■ 异步性

- 由于资源等因素的限制，程序的执行通常都不可能“一气呵成”，而是以“停停走走”的方式运行



OS特征——并发

- **并发 (Concurrency) : 指处理多个同时性活动的能力**
- **由于并发会引起很多问题**
 - 活动切换, 保护, 相互依赖的活动间同步
- **在计算机系统同时存在多个程序运行, 单个CPU**
 - 宏观上: 这些程序同时在执行
 - 微观上: 任何时刻只有一个程序真正在运行, 即这些程序在CPU上是轮流执行的
- **并行 (Parallel) : 与并发相似, 是指多个不同程序同时在多个硬件部件上执行**



■ 共享 (Sharing)

- 操作系统与多个用户的程序共同使用计算机系统中的资源
(共享有限的系统资源)

操作系统要对系统资源进行合理分配和使用
资源在一个时间段内交替被多个进程使用

- 互斥共享 (如打印机)
- 同时共享 (如可重入代码, 磁盘文件)

问题: 资源分配难以达到最优化, 如何保护资源



OS特征——虚拟

■ 虚拟 (Virtual)

- 一个物理实体映射为若干个对应的逻辑实体——分时或分空间
- 虚拟是操作系统管理系统资源的重要手段，可提高资源利用率

■ CPU虚拟：每个进程的“虚处理机”

■ 存储器虚拟：每个进程都有独立的虚拟地址空间 (代码+数据+堆栈)



OS特征——异步性

■ 异步

➤ 系统中并发执行的多道程序“走走停停”即随时可能面对中断，以不可预知的速度向前推进

- 在多道程序环境下，允许多个程序并发执行，但由于资源有限，进程的执行不是一贯到底
- 进程的执行顺序和执行时间不确定



操作系统引论

- 什么是操作系统
- 为什么学习操作系统
- 怎么学习操作系统
- 操作系统简史
- 操作系统功能与特征
- **操作系统的结构**
- 计算机硬件与操作系统机制

■ 复杂系统的构建必须考虑其内部结构

- 不同目标之间往往存在冲突
- 不同需求之间需要进行权衡

■ 操作系统中的"瓦萨号"

- 1991-1995年，IBM投入20亿美元打造Workplace操作系统
- 目标过于宏伟，系统过于复杂，导致项目失败
- 间接导致IBM全力投入扶植Linux操作系统





操作系统的不同目标

■ 用户目标

- 方便使用
- 容易学习
- 功能齐全
- 安全
- 流畅
-

■ 系统目标

- 容易设计、实现
- 容易维护
- 灵活性
- 可靠性
- 高效性
- ...



降低操作系统复杂性

■ 重要设计原则：策略与机制的分离

- 策略（Policy）：要做什么 —— 相对动态
- 机制（Mechanism）：怎么做 —— 相对静态
- 操作系统可仅通过调整策略来适应不同应用的需求

例子	策略	机制
登录	什么用户、以什么权限登录	输入处理、策略文件管理、桌面启动加载 ...
调度	调度算法：Round-robin、Earliest Deadline First ...	调度队列、调度实体（如线程）的表示、调度中断处理 ...

操作系统的结构及演进

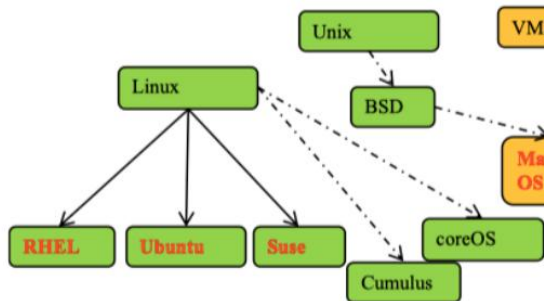
Monolithic kernel (宏内核) : 一个单一庞大的内核负责资源管理；统一系统调用层处理所有OS服务；高耦合，低可靠。

Microkernel (微内核) : 内核只负责IPC，模块化好，高可靠性，IPC成为性能关键

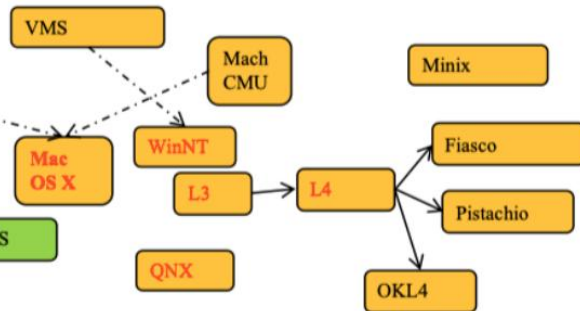
Exokernel : 资源管理和保护隔离，应用负责资源管理

Multikernel : 通过多内核来管理异构多核设备

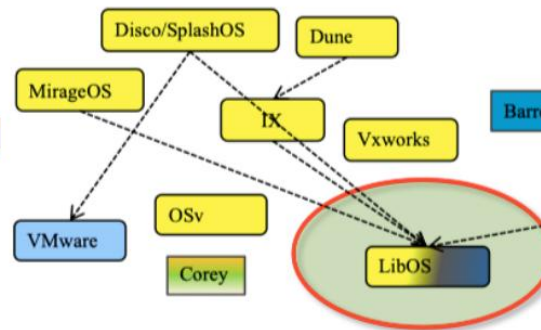
Monolithic (宏内核) 1960s



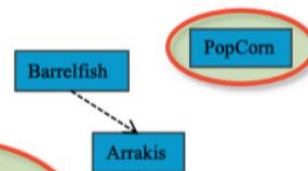
Microkernel(微内核) 1980s



Exokernel 1990s



Multikernel 2010s

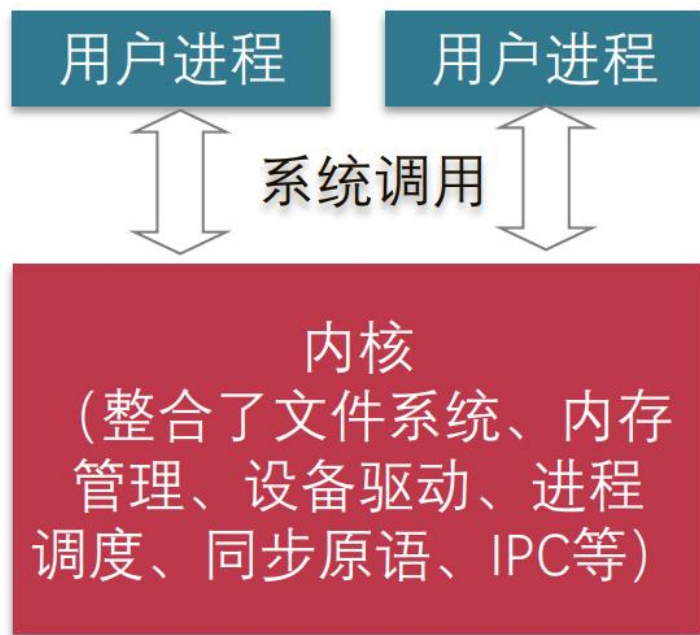




宏内核 (Monolithic Kernel)

■ 整个系统分为内核与应用两层

- 内核：运行在特权级，集中控制所有计算资源
- 应用：运行在非特权级，受内核管理，使用内核服务





宏内核的优缺点分析

■ 宏内核拥有丰富的沉淀和积累

- 拥有巨大的统一的社区和生态
- 针对不同场景优化了30年

■ 宏内核的结构性缺陷

- 安全性与可靠性问题：模块之间没有很强的隔离机制
- 实时性支持：系统太复杂导致无法做最坏情况时延分析
- 系统过于庞大而阻碍了创新：Linux代码行数已经过2千万



宏内核难以满足的场景

■ 向上向下的扩展

- 很难去剪裁/扩展一个宏内核系统支持从KB级别到TB级别的场景

■ 硬件异构性

- 很难长期支持一些定制化的方式去解决一些特定问题

■ 功能安全

- 一个广泛共识：Linux无法通过汽车安全完整性认证（ASIL-D）

■ 信息安全

- 单点错误会导致整个系统出错，而现在有数百个安全问题（CVE）

■ 确定性时延

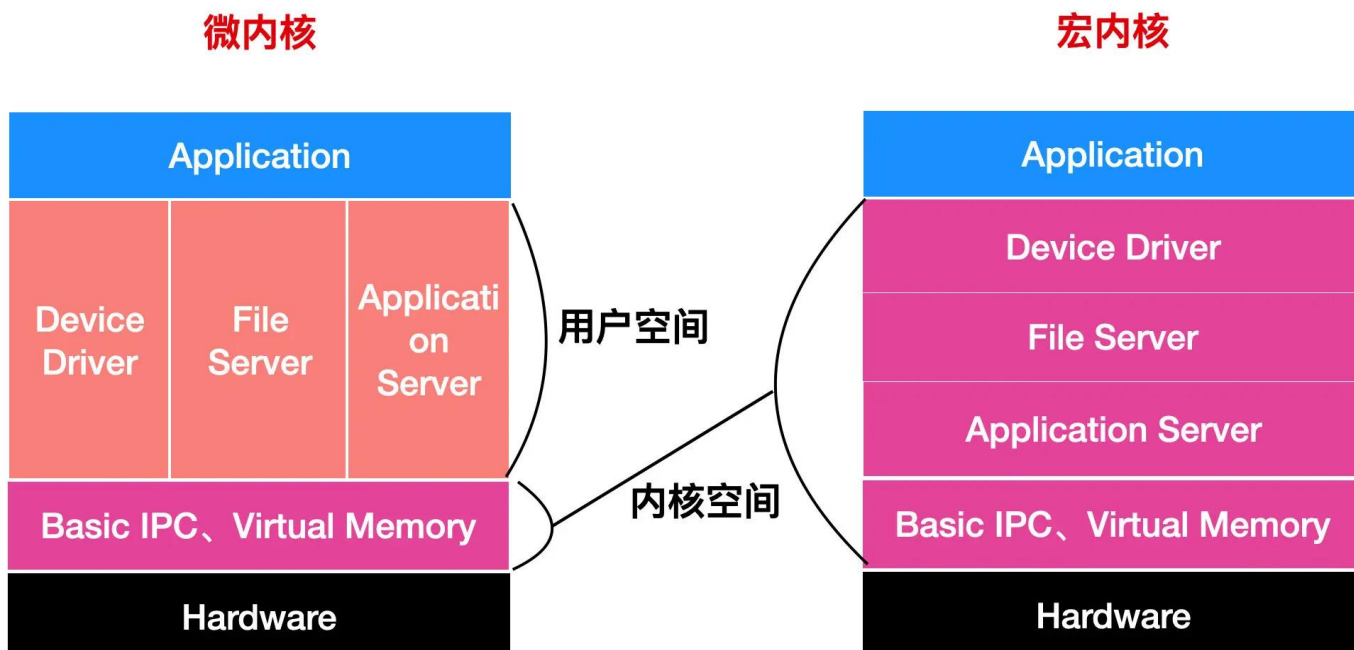
- Linux花费10+年合并实时补丁，目前依然不确定是否能支持确定性时延



微内核 (Monolithic Kernel)

■ 设计原则：最小化内核功能

- 将操作系统功能移到用户态，称为"服务" (Server)
- 在用户模块之间，使用消息传递机制通信



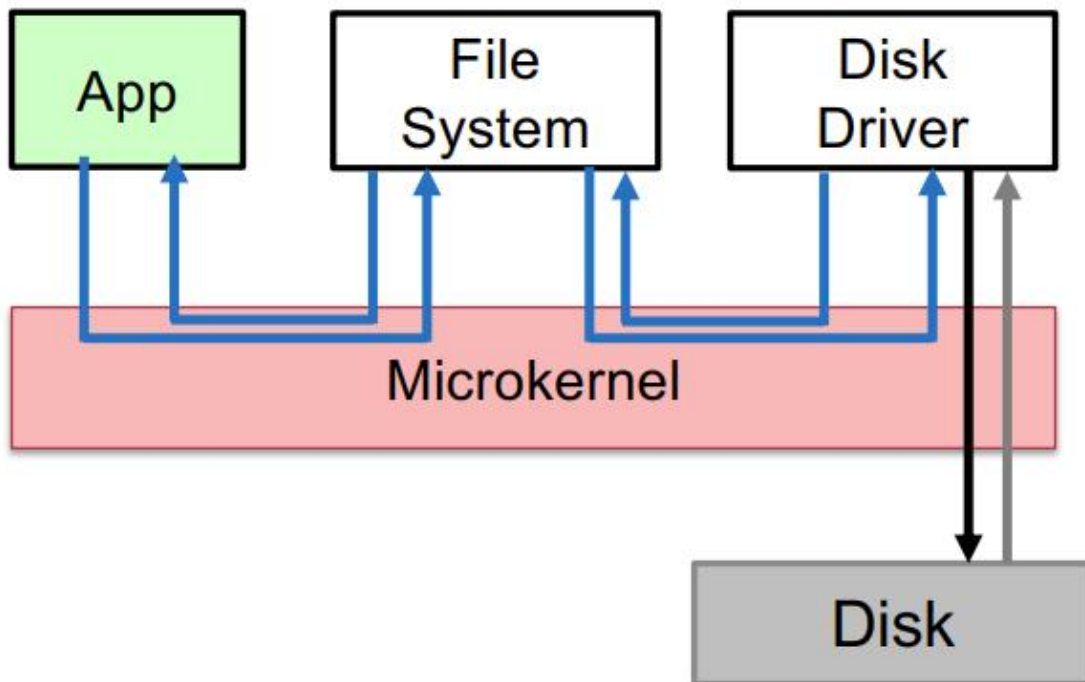
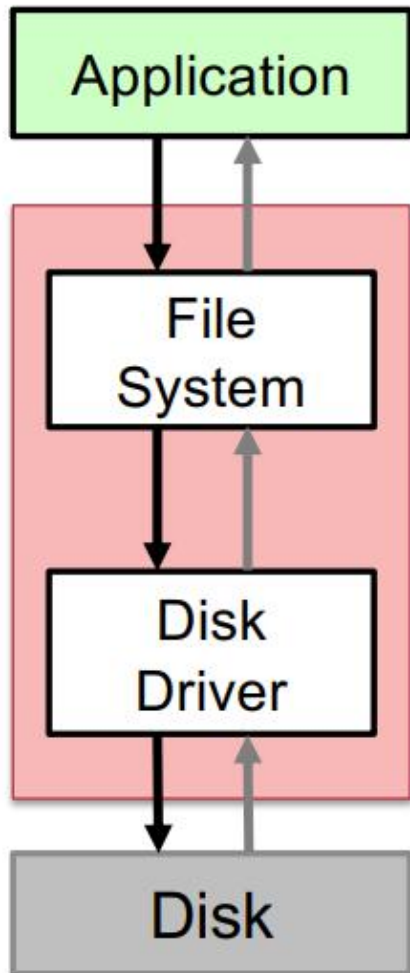


微内核结构

- 内核中只包括中断处理、进程通信（IPC）、基本调度等
- 文件系统、网络功能、内存管理、设备管理等作为服务在微内核上运行
- 优点：
 - 内核易于实现、可移植性好、配置灵活、适应分布式环境（本地内核与远程内核对服务同样的支持）
- 缺点：
 - 速度较慢。（扩大内核减少切换；减少内核提高其他优点）



例：文件的创建



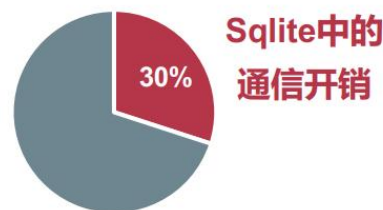
微内核的优缺点分析

■ 优点

- 易于扩展：直接添加一个用户进程即可为操作系统增加服务
- 易于移植：大部分模块与底层硬件无关
- 更加可靠：在内核模式运行的代码量大大减少
- 更加安全：即使存在漏洞，服务与服务之间存在进程粒度隔离
- 更加健壮：单个模块出现问题不会影响到系统整体

■ 缺点

- 性能较差：内核中的模块交互由函数调用变成了进程间通信
- 生态欠缺：尚未形成像Linux一样具有广泛开发者的社区





Mach微内核

■ 1985年, Mach 发布

- 由CMU开发, Rick Rashid领导
- 对操作系统发展产生了重大影响

■ 1986年, Mach 2.5 (性能比UNIX差25%)

- 包含大量BSD的代码, 如1:1的task与process映射, 导致内核比UNIX更大
- 取得了商业成功, 用于NeXT, 最终被苹果收购

■ 1990年, Mach 3.0 (性能比UNIX差67%)

- 规避法律风险, 去掉了BSD的代码, 重写了IPC以提高性能
- 提出"continuation", 为用户态应用提供了更多控制
 - 允许应用自己在切换的时候保存/恢复上下文, 进一步减小microkernel



Mach实现了哪些功能?

■ 任务和线程管理

- 任务，是资源分配的基本单位；线程，是执行的基本单位
- 对应用提供调度接口，应用程序可实现其自定义的调度策略

■ 进程间通信（IPC）：通过端口（port）进行通信

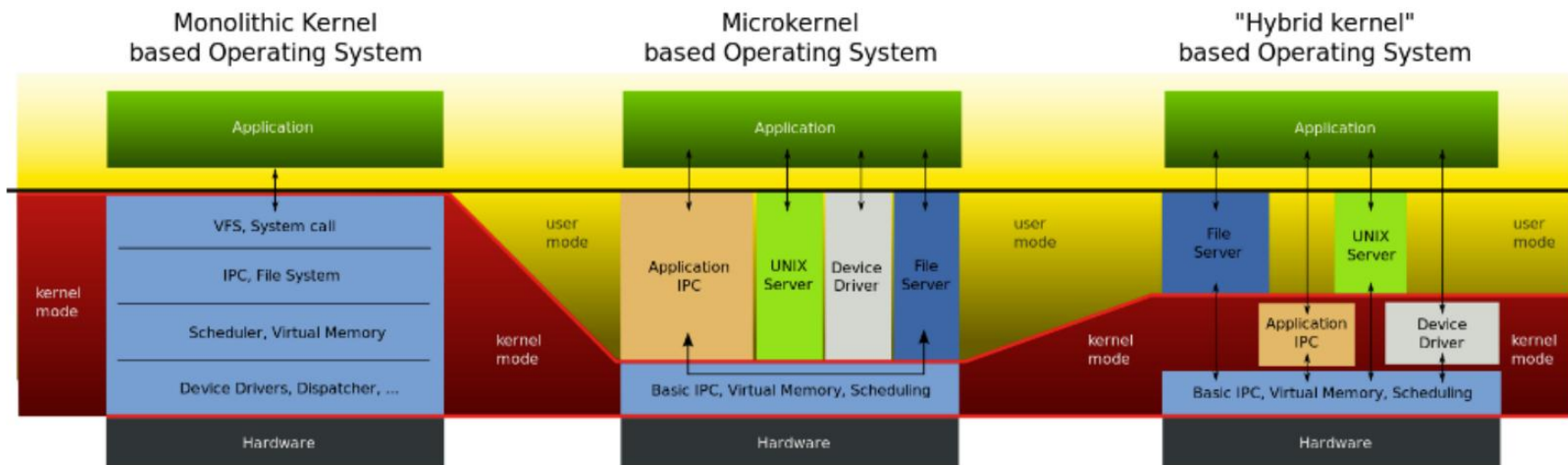
■ 内存对象管理：虚拟内存

■ 系统调用重定向：允许用户态处理系统调用

- 支持对系统调用的功能扩展，例如，二进制翻译、跟踪、调试等

■ 宏内核与微内核的结合

- 将需要性能模块重新放回内核态
- macOS / iOS: Mach微内核 + BSD 4.3 + 系统框架
- Windows NT: 微内核 + 内核态的系统服务 + 系统框架





Windows NT

■ Integral子系统（用户态）

- 负责处理I/O、对象
- 管理、安全、进程等

■ 环境子系统（用户态）

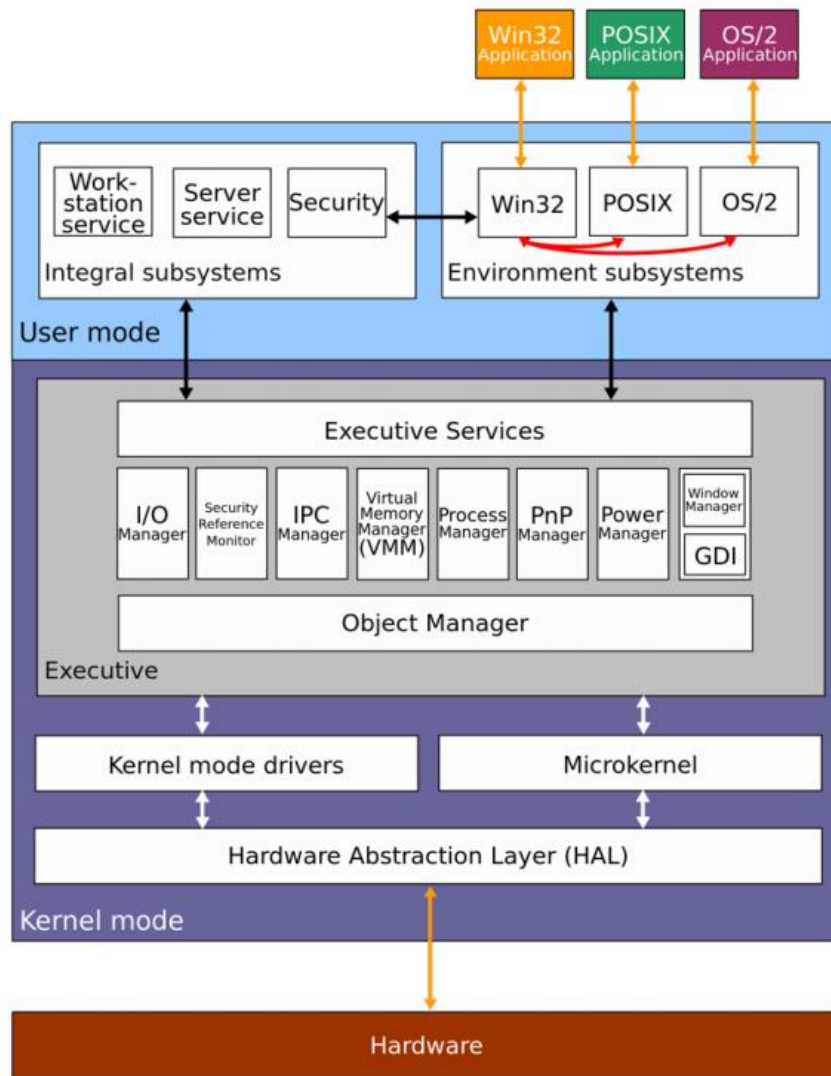
- POSIX

■ Executive（内核态）

- 为用户态子系统提供服务

■ Microkernel

- 提供进程间同步等功能



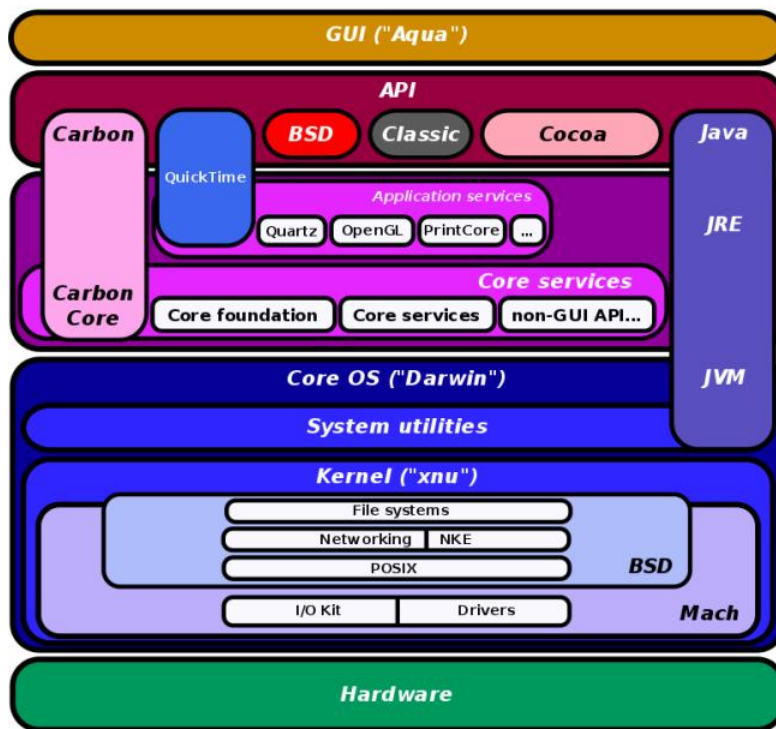


macOS

■ XNU内核

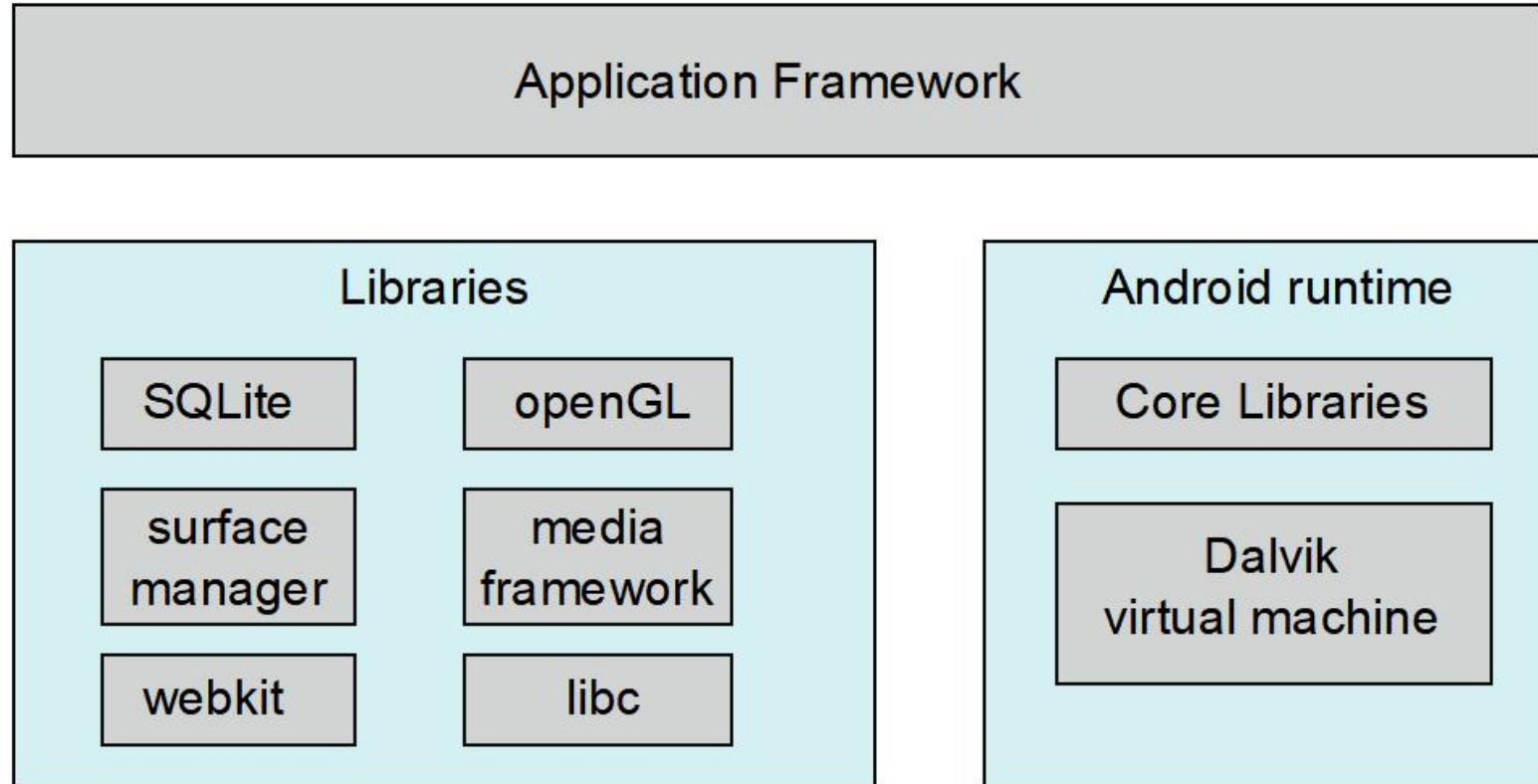
- 基于Mach-2.5打造
- BSD代码提供文件系统、网络、POSIX接口等

■ macOS与iOS





Android Architecture





外核架构 (Exokernel)

■ Exokernel 不提供硬件抽象

- "只要内核提供抽象，就不能实现性能最大化"
- 只有应用才知道最适合的抽象 (end-to-end原则)

■ Exokernel 不管理资源，只管理应用

- 负责将计算资源与应用的绑定，以及资源的回收
- 保证多个应用之间的隔离

回顾：操作系统 = 服务应用 + 管理应用

内核态：Exokernel

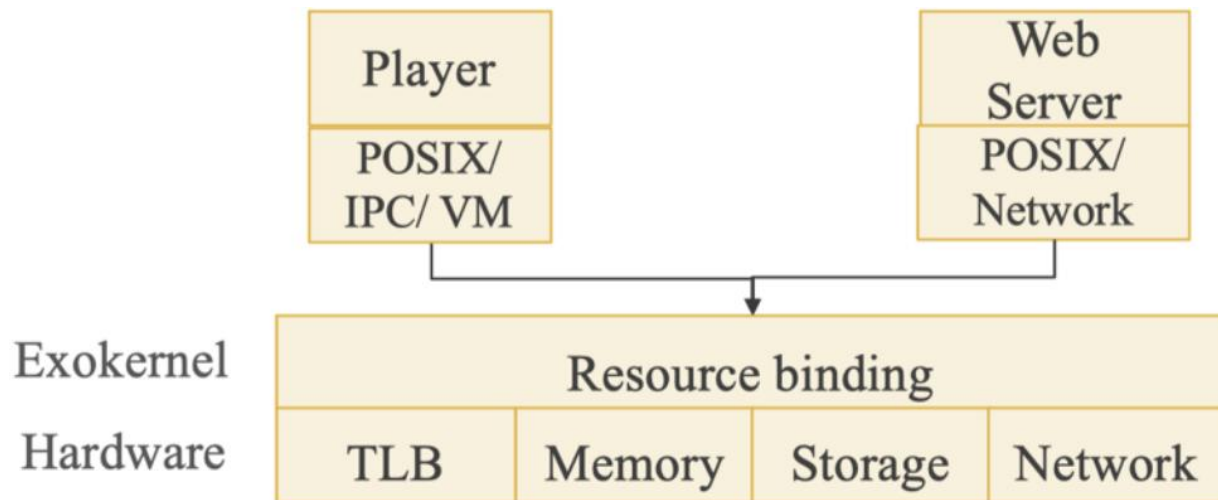
用户态：libOS



Exokernel + LibOS

■ 库OS (LibOS)

- 策略与机制分离：将对硬件的抽象以库的形式提供
- 高度定制化：不同应用可使用不同的LibOS，或完全自定义
- 更高性能：LibOS与应用其他代码之间通过函数调用直接交互





不同操作系统架构的对比

